

Evolution status after Jacksonville 2018

Abstract

This paper is a collection of items Evolution has worked on up to and in the Jacksonville meeting, their status, and plans for the future.

Concepts

P0782R0 "A Case for Simplifying/Improving Natural Syntax Concepts" was reviewed in an evening session in Jacksonville, further work and revision is expected before C++20 is finalized.

P0745R0 "Concepts in-place syntax" was also reviewed in an evening session in Jacksonville. Further work and revision is expected before C++20 is finalized. A non-template declaration syntax for constrained declarations is being worked on with the attempt to have it in C++20; the guidance from the committee is that templates need to be syntactically different from non-templates.

Coroutines

D0973R0 "Coroutines TS Use Cases and Design Issues" was reviewed, and we expect to have a more concrete proposal in Rapperswil. The current expectation is that we'll try to ship coroutines in C++20.

P0913R0 "Add symmetric coroutine control transfer" and P0914R0 "Add parameter preview to coroutine promise constructor" were approved in Jacksonville and moved in Jacksonville, for the Coroutines TS.

Modules

P0947R0 "Another take on Modules" was reviewed. Ship vehicles of changes were discussed, and the guidance is that we are looking at merging the proposed changes into the Modules TS design, instead of going with a separate TS. It's expected that there will be a proposal for such a merge for Rapperswil, and it's also possible that we will target C++20 with the merged approach.

We are also going to investigate ways to avoid code breakage caused by introduced keywords. There's no consensus to make 'module' a context-sensitive keyword, nor is there consensus to take a different keyword.

Contracts

Evolution discussed a clarification to what the result of observable side-effects in contract pre/post-conditions is, and the guidance is that it's Undefined Behavior.

Reflection and related facilities

P0670R2 "Static reflection of functions" was reviewed and approved.

P0784R1 "Standard containers and constexpr" was reviewed and approved.

P0732R0 "Class Types in Non-Type Template Parameters" was reviewed and approved. P0424 is withdrawn; some parts of it will be folded into P0732.

C++ Stability, Velocity, and Deployment Plans

P0684R2 was reviewed in Jacksonville. Evolution supports turning it into a Standing Document; the expectation is that the document will be revised and reviewed again. Winters will be the main author, Dos Reis, Stroustrup and Spertus will assist.

Standard Library Compatibility Promises

P0921R0 was reviewed in Jacksonville. Evolution supports turning it into a Standing Document and to add the promises also to the Library Introduction clause, [library]. Winters will be the main author, Hedberg, Liber, Stroustrup, Dennett, Brown, Nishanov, Meredith, Spertus, Steagall and Vandevoorde will assist.

LEWG wishlist for EWG

P0922R0 was reviewed in Jacksonville. There is ongoing work in some of the areas mentioned in the paper, but there's going to be a separate status quo analysis for macros; what the remaining use cases for macros are, what the status of the non-macro solutions for those problems is and so forth, led by Voutilainen, assisted by Stroustrup, Dos Reis, Dennett and Romeo.

P0934R0 "A Modest Proposal: Fixing ADL" was reviewed, further work, implementation and analysis is expected, there is no ship vehicle designation at this time.

Fixes for range-for and structured bindings

P0962R0 relaxes the range-for customization point lookup for begin/end so that members are used if both members are found, and if both are not found, ADL begin/end are looked up and used. This opens the door for making iostreams range-for-traversable, and allows user wrappers of iostreams to be made so traversable. This fix is moved as a DR in Jacksonville.

P0961R0 relaxes the structured bindings customization point lookup so that a member get is required to be a template. This allows wrappers of smart pointers and tuples to be decomposed. This fix is moved as a DR in Jacksonville.

D0969R0 allows structured bindings of accessible members, not just public members. This fix is moved as a DR in Jacksonville.

P0931R0 "Structured bindings with polymorphic lambdas" and P0963R0 "Structured binding declaration as a condition" were rejected.

Changes to relational operators and the spaceship operator

P0946R0 "Towards consistency between <=> and other comparison operators" was reviewed; the direction is towards multiple deprecations and incompatible changes even without deprecation. The ship vehicle for these changes hasn't been decided yet, and we expect more data to be gathered before Rapperswil.

"relational operators give mathematical meaning on arithmetic types (breaking C, C++17 compatibility) with an unspecified ship vehicle? 5 | 17 | 5 | 4 | 0"

P0905R0 "Symmetry for spaceship" was reviewed and approved.

utf-8

P0482R1 "char8_t: A type for UTF-8 characters and strings" was reviewed. No consensus for a TS, our target is IS, and the proposal is approved to go to Core for wording review.

Initialization

P0960R0 "Allow initializing aggregates from a parenthesized list of values" was reviewed. There's a somewhat related issue about aggregates with deleted constructors. We expect further work to appear in Rapperswil, either with a combined aggregate initialization proposal or with the deletedness issues separated out.

Two's complement

P0907R0 D0907r1 "Signed Integers are Two's Complement" was reviewed. It's not ready for Core yet, we expect further work to appear in Rapperswil.

Feature macros

The feature macro proposal was not reviewed, it will return in Rapperswil, with the aim to incorporate into C++20.

Speculation attacks

P0928R0 "Mitigating Speculation Attacks in C++" was reviewed, we expect further work to appear in Rapperswil.

Other papers

P0892R0 "explicit(bool)" was reviewed and approved.

P0929R0 "Checking for abstract class types" was reviewed and approved.

A fairly large amount of unreviewed papers, will see if those can be reviewed later, the focus is on the major features, Concepts, Modules, Coroutines.